

رسالة محمد



یادگیری ماشین

مدرس: محمدرضا محمدی

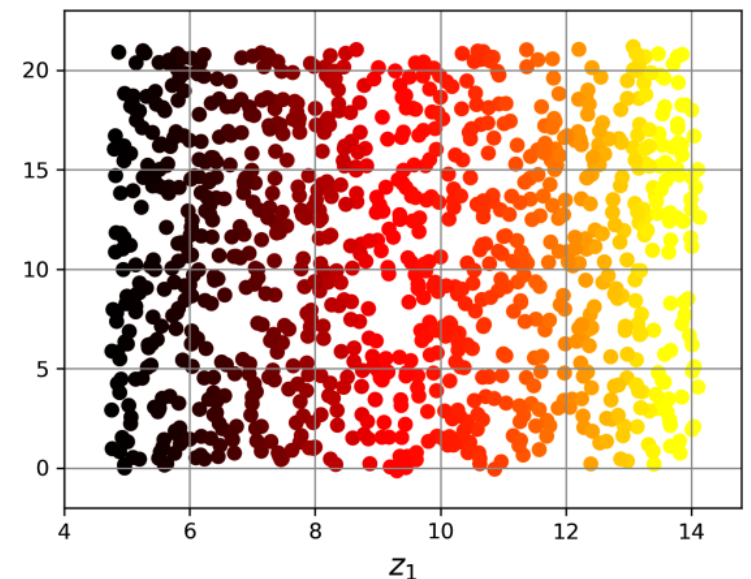
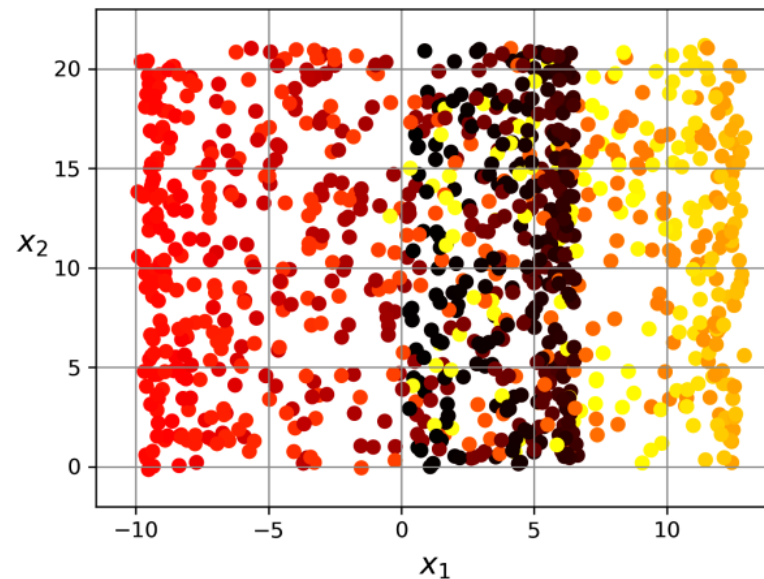
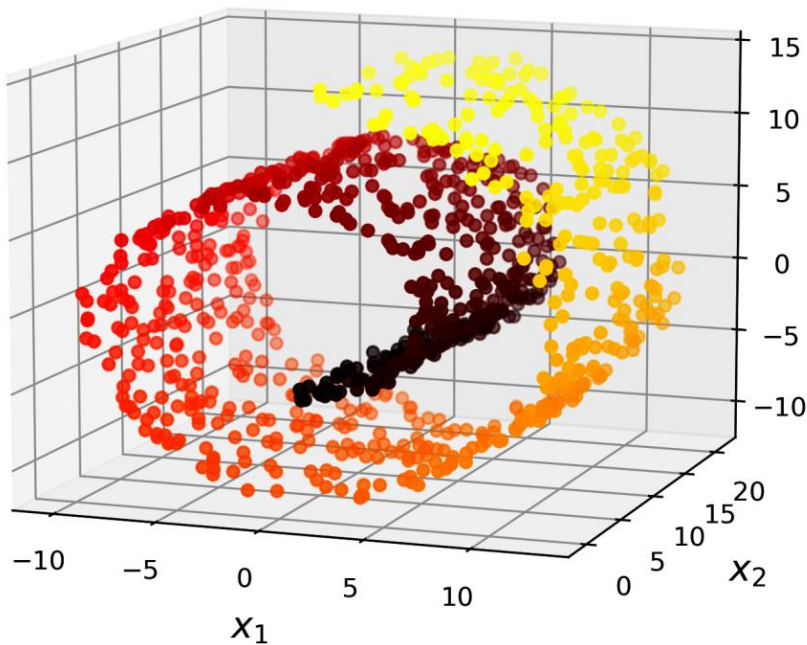
بهار ۱۴۰۲

کاهش بعد

Dimensionality Reduction

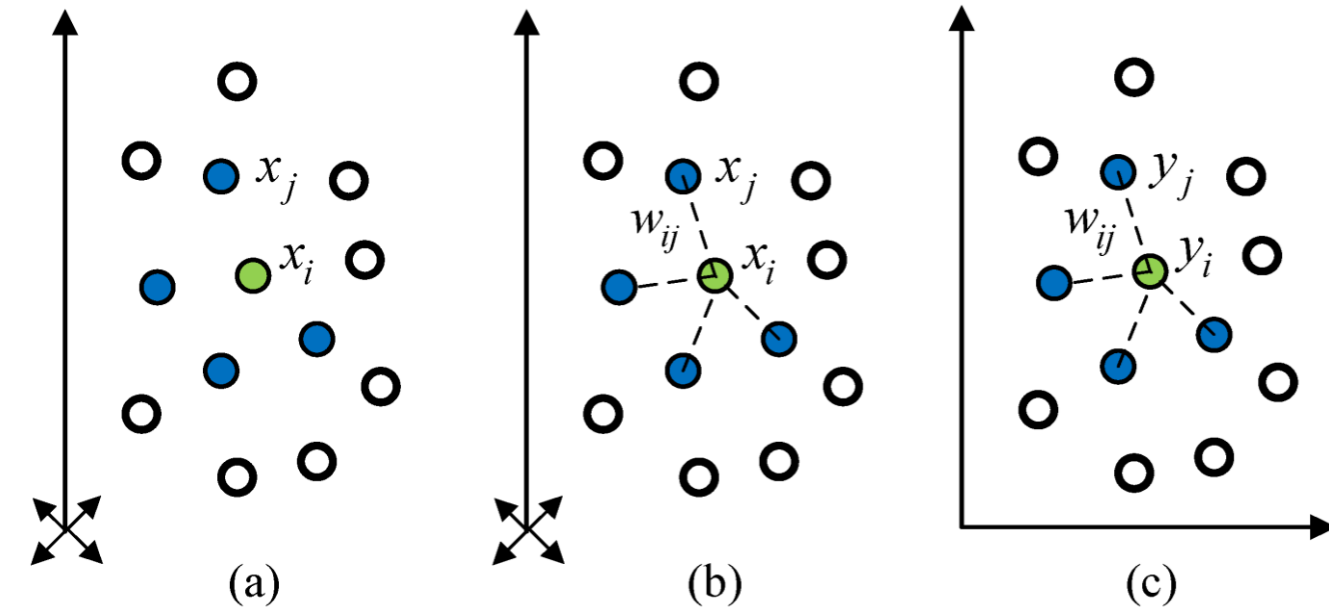
یادگیری منیفولد (Manifold)

- کاهش بُعد خطی نمی‌تواند برای همه مسائل بهینه باشد
- ممکن است نمونه‌هایی که فاصله زیادی از هم دارند، بسیار شبیه به هم شوند
- در یادگیری منیفولد، به دنبال فضایی غیرخطی هستیم که بتواند تفاوت‌های نمونه‌ها را به خوبی حفظ کند



LLE (جانمایی خطی محلی)

- ایده Locally Linear Embedding این است که نزدیکترین همسایه‌های (nearest neighbors) مربوط به هر نمونه را پیدا کند و تلاش می‌کند تا زیرفضایی غیرخطی با ابعاد پائین را بیابد که ارتباط محلی نمونه‌ها تا حد امکان حفظ شود

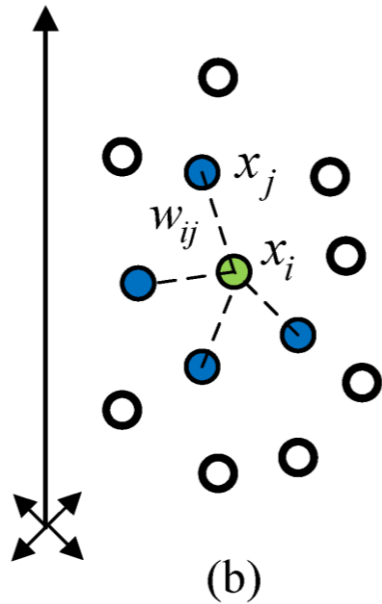
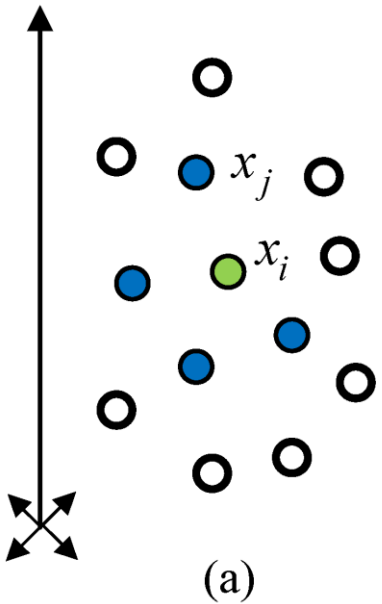


- برای هر نمونه آموزشی $\mathbf{x}^{(i)}$ ، تعداد k نزدیکترین همسایه مشخص شده و برای هر کدام وزنی محاسبه می‌شود که $\sum w_{i,j} \mathbf{x}^{(j)}$ نزدیک به $\mathbf{x}^{(i)}$ باشد

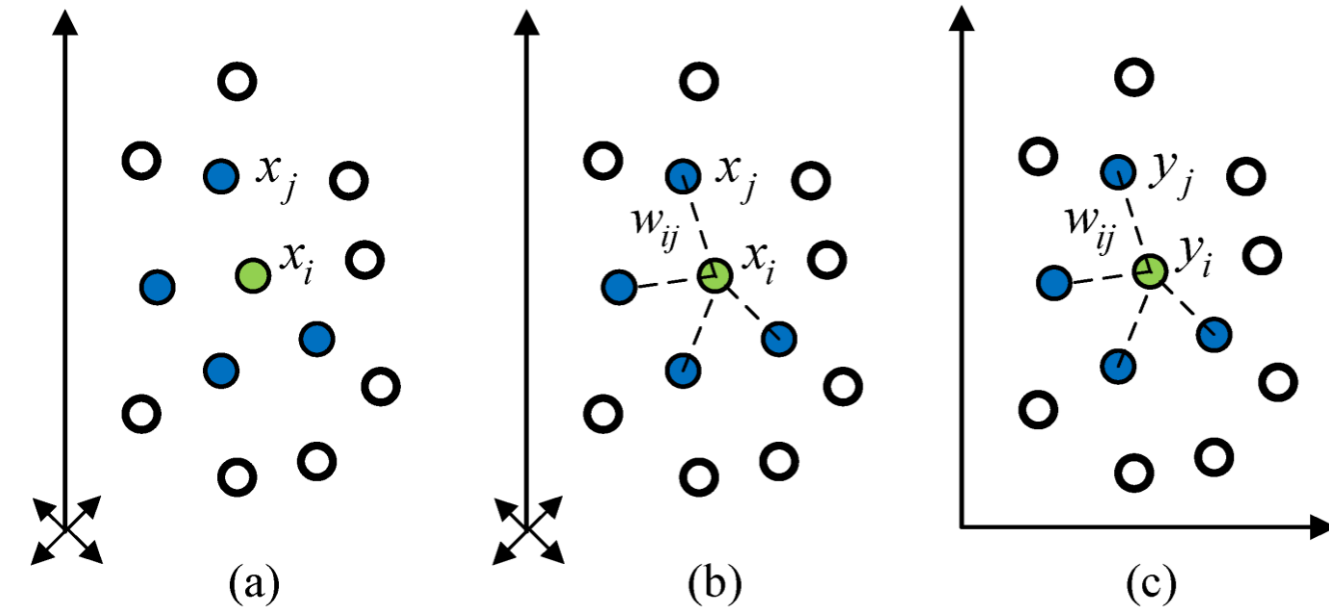
$$\widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2$$

$$\text{subject to } \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ n.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^m w_{i,j} = 1 & \text{for } i = 1, 2, \dots, m \end{cases}$$

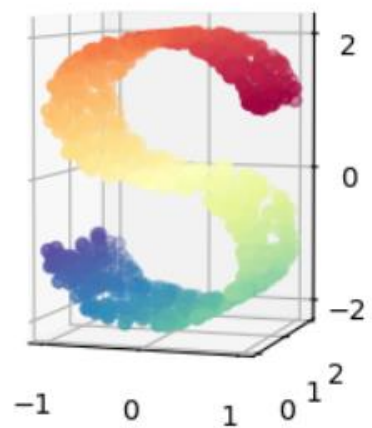
- ماتریس $\widehat{\mathbf{W}}$ ارتباط خطی محلی بین نمونه‌های آموزشی را مدل می‌کند



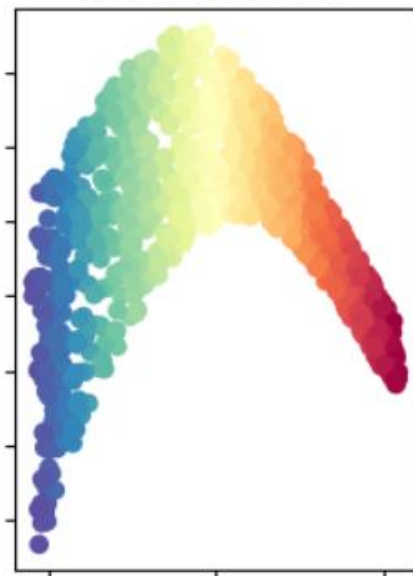
- در گام دوم، نمونه‌های آموزشی به زیرفضای d بعدی نگاشت می‌شوند با این شرط که ارتباط خطی محلی بین نمونه‌ها تا حد امکان حفظ شود
- اگر $\mathbf{z}^{(i)}$ نسخه کاهش بعد یافته $\mathbf{x}^{(i)}$ باشد، می‌خواهیم فاصله آن با $\sum \hat{w}_{i,j} \mathbf{z}^{(j)}$ تا حد امکان کم باشد



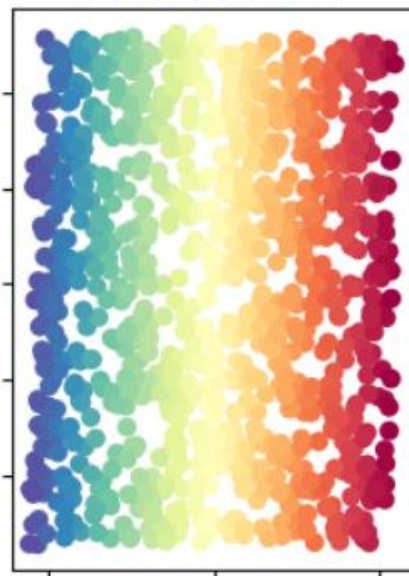
$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$



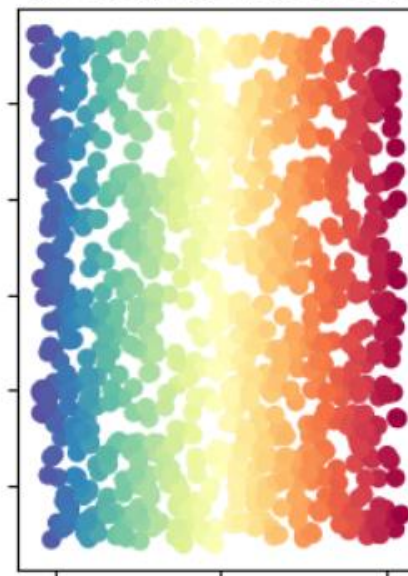
LLE (0.083 sec)



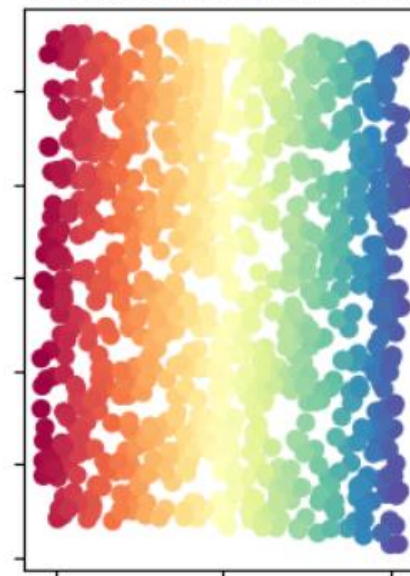
LTSA (0.12 sec)



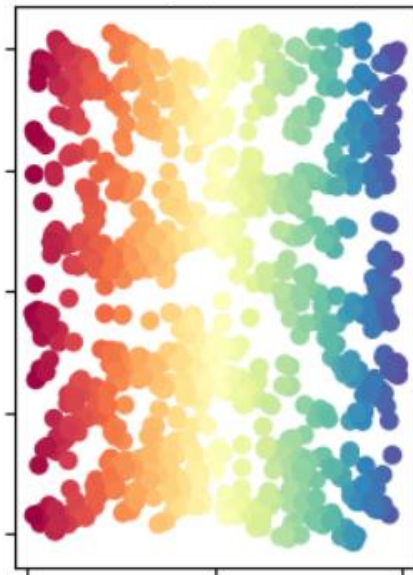
Hessian LLE (0.24 sec)



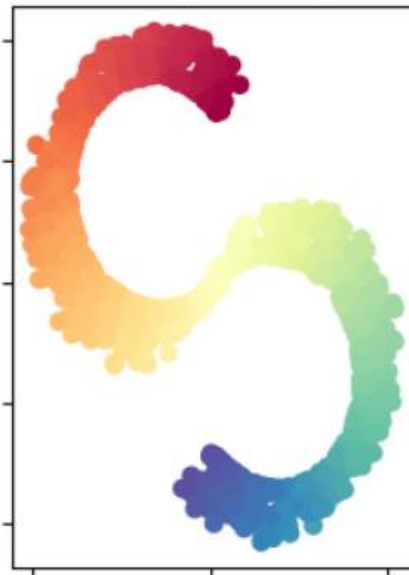
Modified LLE (0.19 sec)



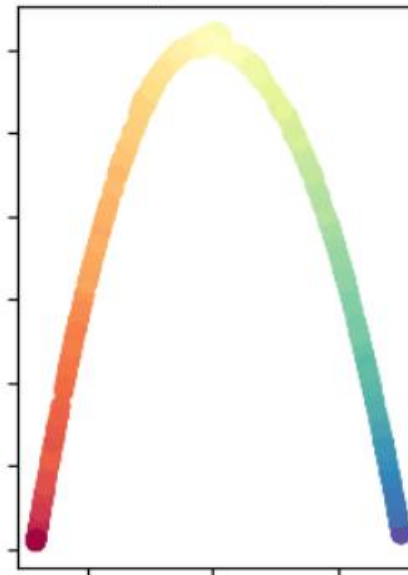
Isomap (0.37 sec)



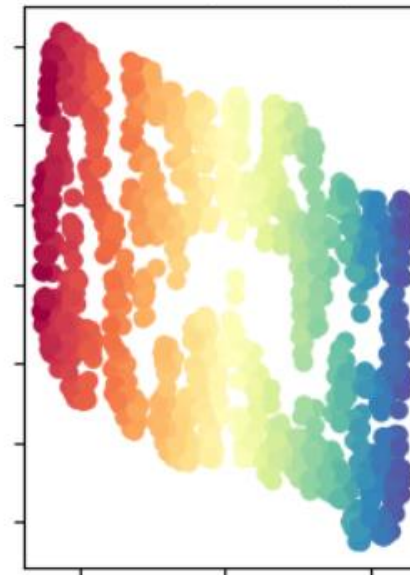
MDS (1.9 sec)



SE (0.071 sec)

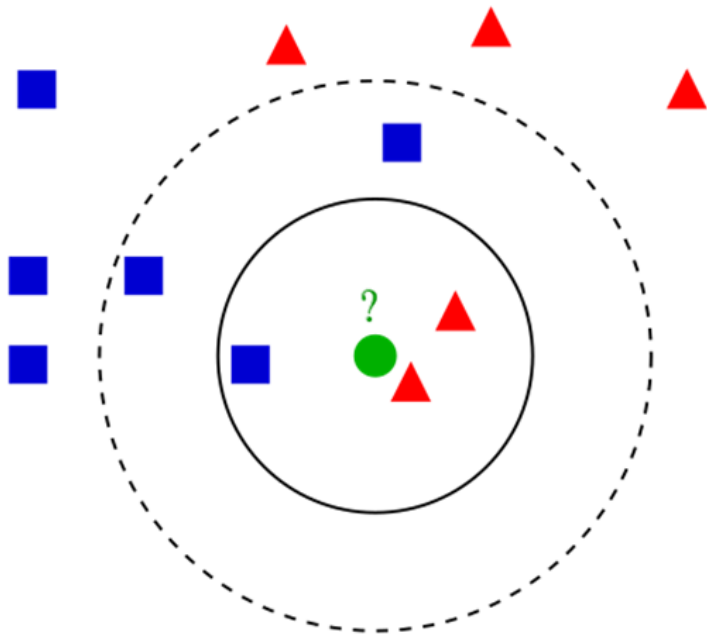


t-SNE (6.7 sec)



k Nearest Neighbors

- kNN یک الگوریتم با نظارت است که از میان نمونه‌های آموزشی، k نمونه نزدیکتر به نمونه آزمون مشخص می‌شوند و تصمیم‌گیری بر اساس آنها انجام می‌شود
- آموزش بسیار سریع است (فقط باید نمونه‌های آموزشی را در حافظه ذخیره کرد و محاسباتی ندارد)
- آزمون می‌تواند بسیار کند باشد زیرا نمونه آزمون باید با تمام نمونه‌های آموزشی مقایسه شود
- اگر معیار سنجش فاصله نمونه‌ها مناسب باشد، می‌تواند دقت مناسبی نتیجه دهد



kNN

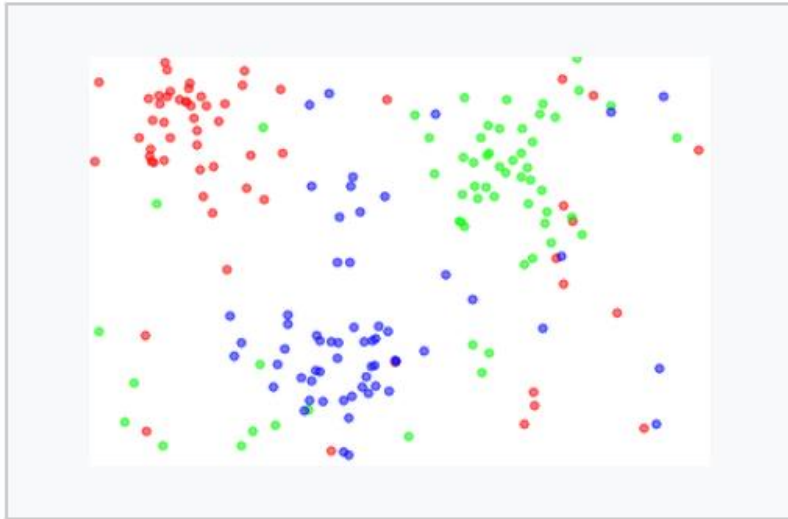


Fig. 1. The dataset.

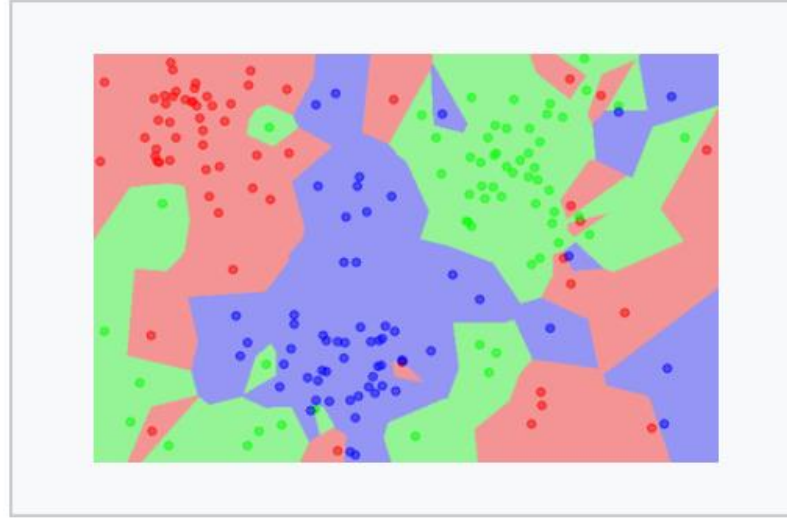


Fig. 2. The 1NN classification map.

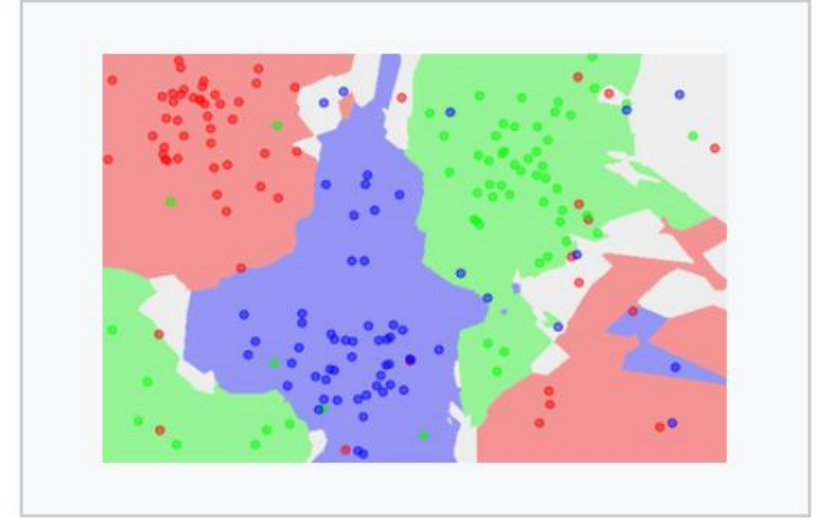


Fig. 3. The 5NN classification map.

الگوریتم‌های خوشه‌بندی

Clustering Algorithms

خوشه‌بندی

- هدف این است که نمونه‌های مشابه درون خوشه‌های مجزا قرار بگیرند
- کاربردهای زیادی در تحلیل داده، سیستم‌های پیشنهاددهنده، متورهای جستجو و ... دارند

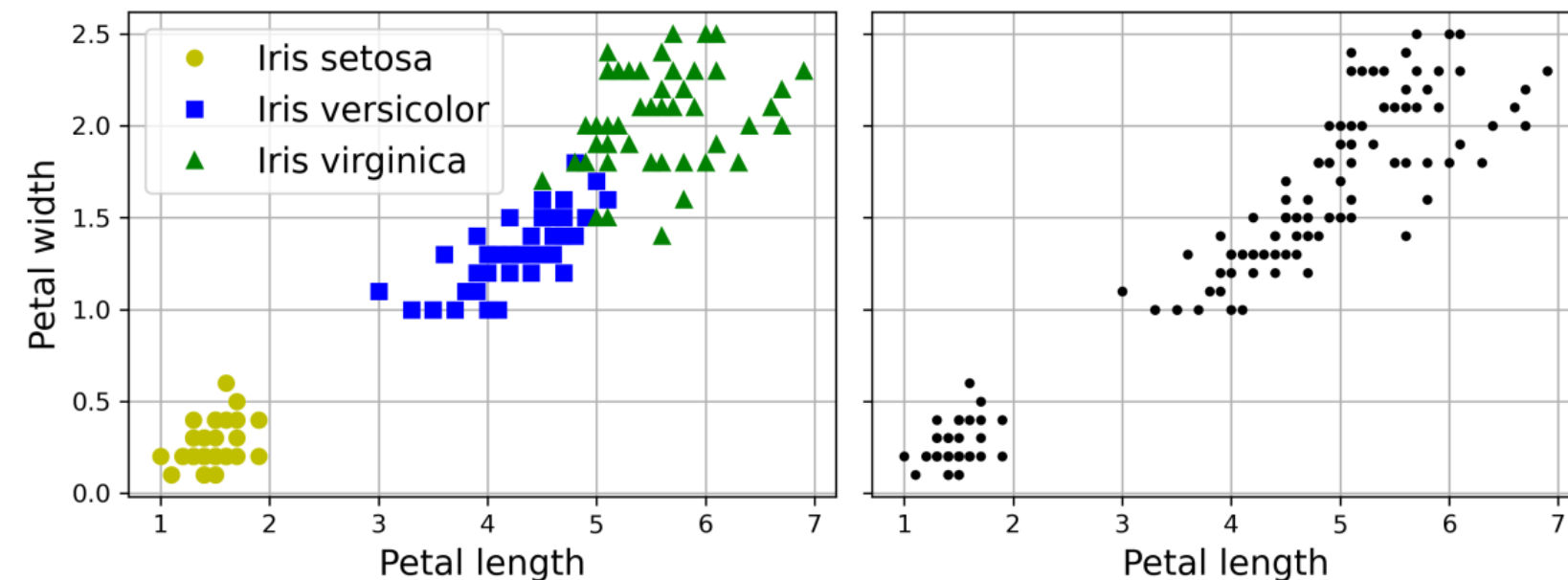


خوشه‌بندی

- در دسته‌بندی، برای هر نمونه برچسبی وجود دارد که مدل باید بتواند آن را برای نمونه‌های جدید پیش‌بینی کند

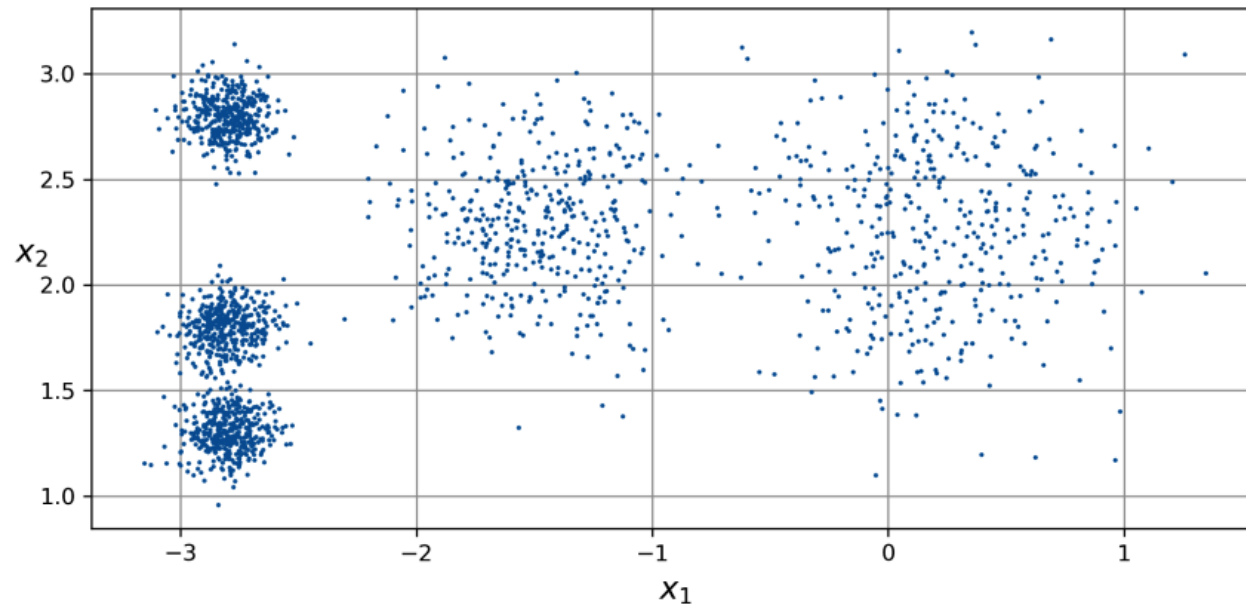
- در خوشه‌بندی، برچسبی برای نمونه‌ها وجود ندارد و مدل باید بتواند نمونه‌های مشابه را شناسایی کند

- در این مثال، با یک خوشه‌بندی ساده و با استفاده از هر ۴ ویژگی، فقط ۵ نمونه از ۱۵۰ نمونه در خوشه اشتباه قرار می‌گیرند



k-means

- به صورت چشمی، در مجموعه داده زیر ۵ خوشه قابل ملاحظه است
- پیش از آنکه جزئیات الگوریتم k-means را بررسی کنیم، نتیجه اجرای آن را ملاحظه می‌کنیم




```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

```
X, y = make_blobs(...) # make the blobs: y contains the cluster IDs, but we
                        # will not use them; that's what we want to predict
```

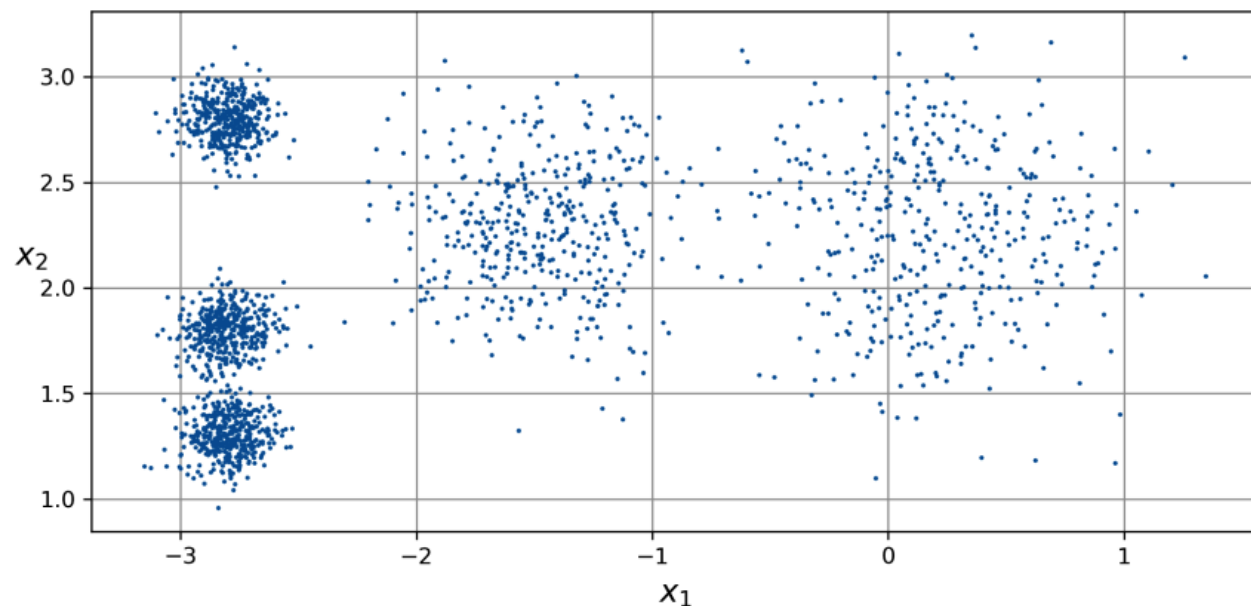
```
k = 5
```

```
kmeans = KMeans(n_clusters=k, random_state=42)
```

```
y_pred = kmeans.fit_predict(X)
```

```
>>> y_pred
```

```
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)
```



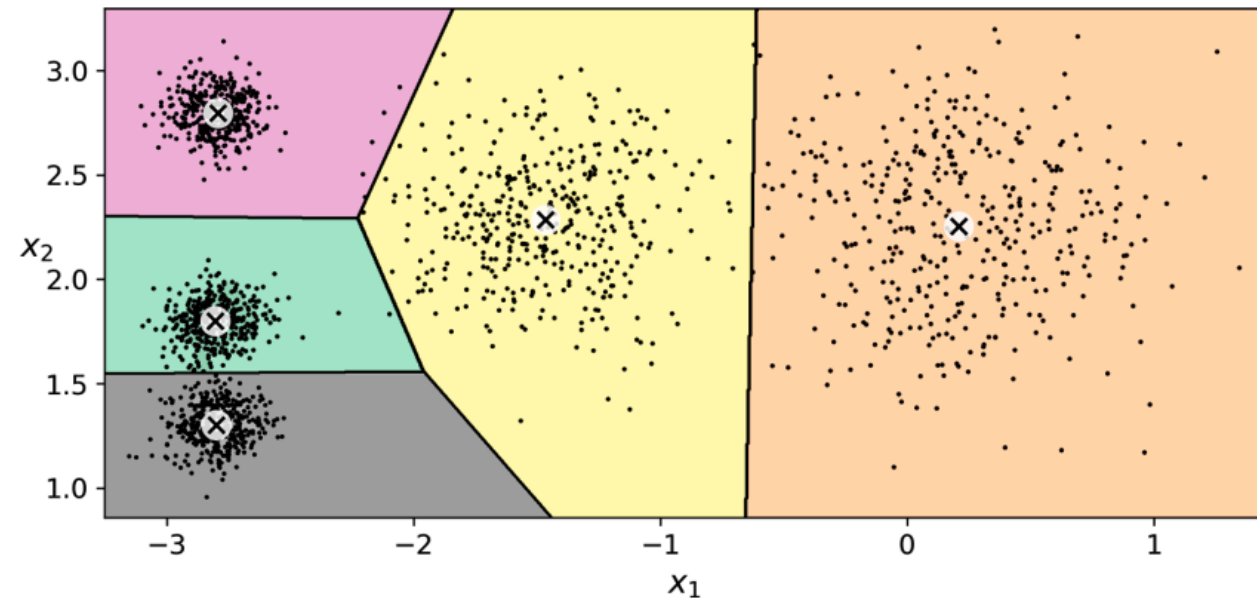
```
>>> kmeans.cluster_centers_
array([[ -2.80389616,  1.80117999],
       [  0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [-2.80037642,  1.30082566]])
```

```
>>> import numpy as np
```

```
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
```

```
>>> kmeans.predict(X_new)
```

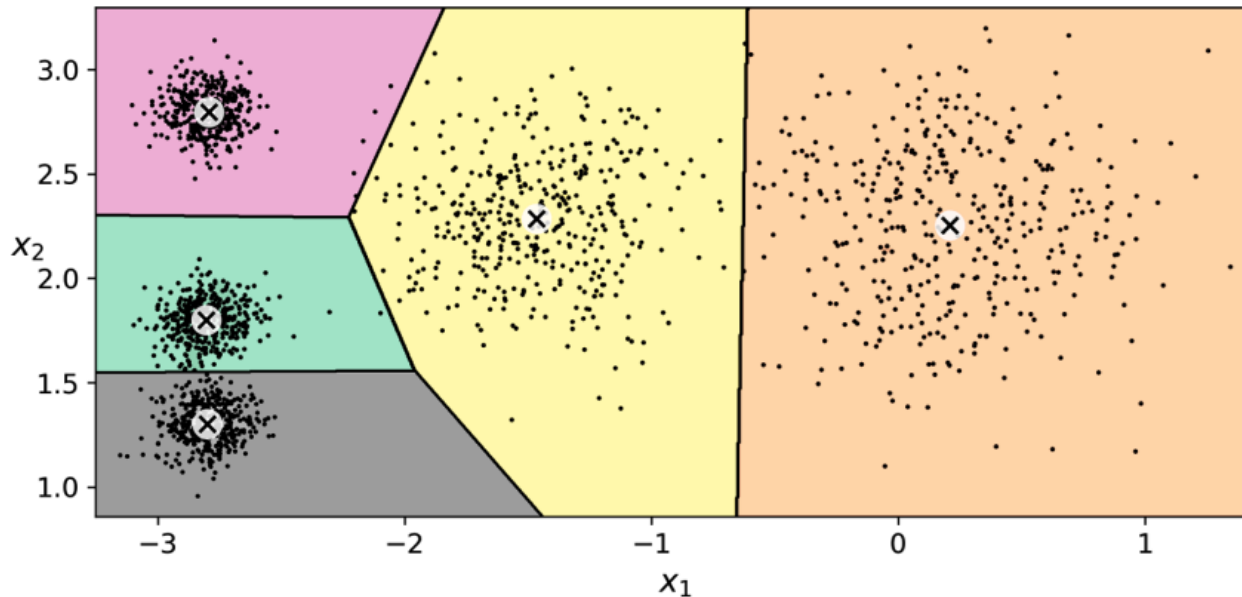
```
array([1, 1, 2, 2], dtype=int32)
```



k-means

- فرض کنید مرکز (centroid) خوشه‌ها را بدانیم، به راحتی می‌توان فاصله هر نمونه با این مراکز را محاسبه کرد و نزدیکترین خوشه را انتخاب کرد

```
>>> kmeans.transform(X_new).round(2)
array([[2.81, 0.33, 2.9 , 1.49, 2.89],
       [5.81, 2.8 , 5.85, 4.48, 5.84],
       [1.21, 3.29, 0.29, 1.69, 1.71],
       [0.73, 3.22, 0.36, 1.55, 1.22]])
```



- متقابلاً، اگر برچسب تمام نمونه‌ها را بدانیم، می‌توان مرکز هر خوشه را با محاسبه میانگین ویژگی‌های تمام نمونه‌های مربوط به آن خوشه بدست آورد
- اما هیچکدام از این دو را نداریم!
- ایده k-means این است که ابتدا مراکز خوشه‌ها را به صورت تصادفی انتخاب کرد و سپس دو گام بالا را تکرار کرد

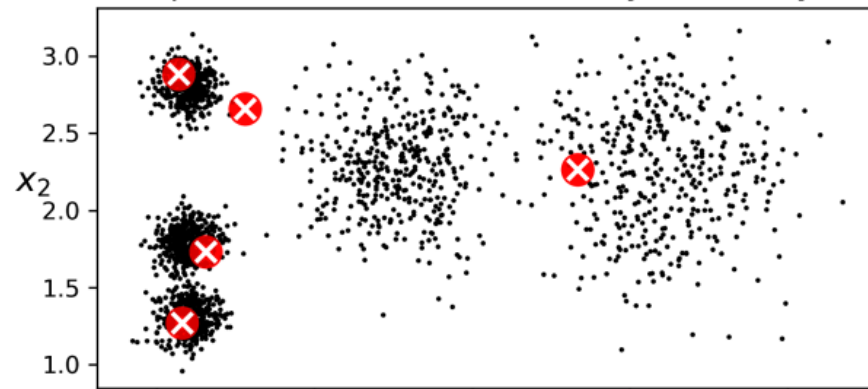
k-means

- انتخاب k نمونه به صورت تصادفی به عنوان مرکز خوشه‌ها
 1. تعیین خوشه نمونه‌ها
 2. به‌روزرسانی مراکز
 3. در صورت عدم همگرایی، رفتن به گام 1
- تضمین می‌شود که در چند کام همگرا خواهد شد

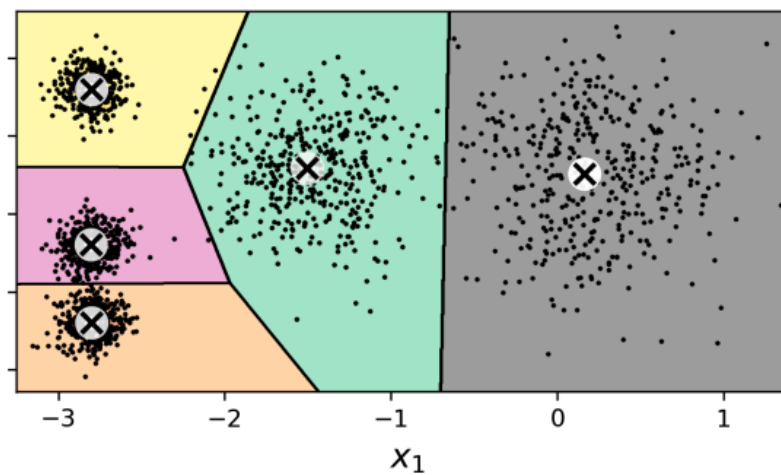
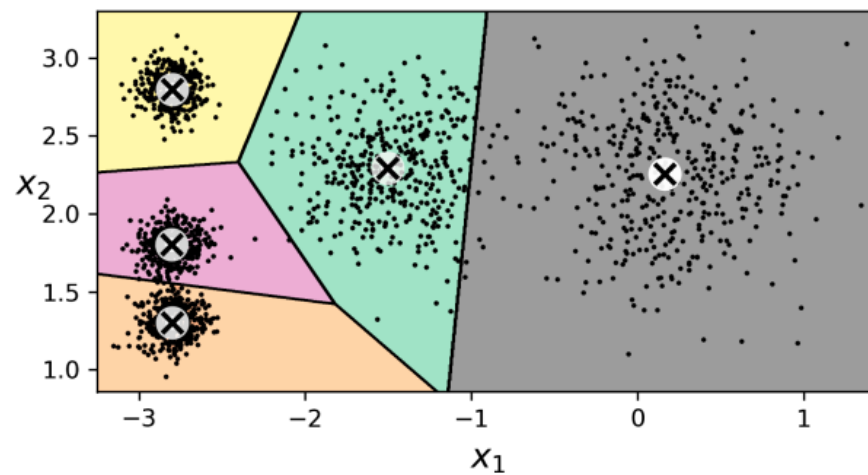
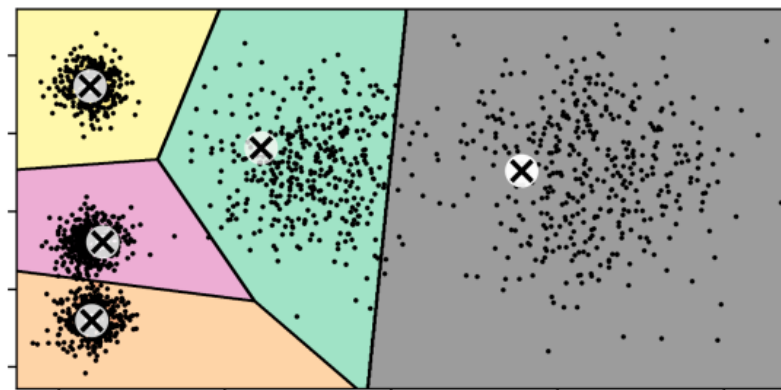
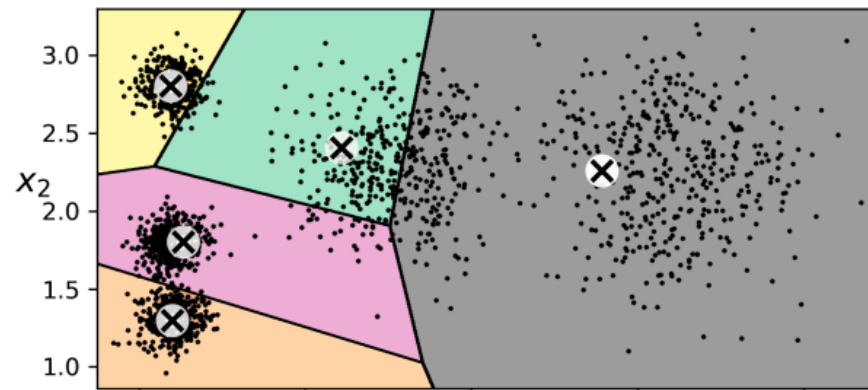
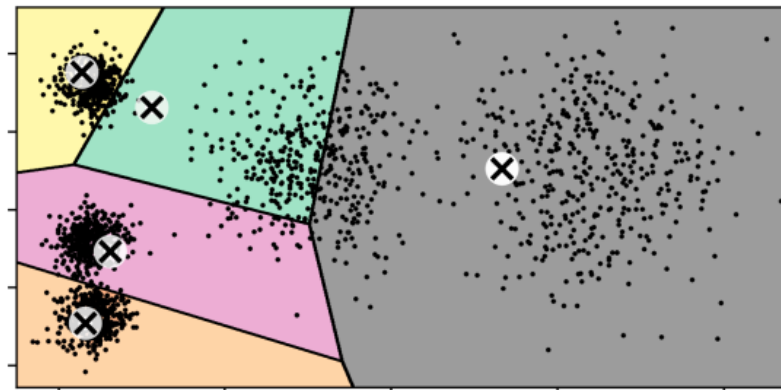
Algorithm 1 k -means algorithm

- 1: Specify the number k of clusters to assign.
 - 2: Randomly initialize k centroids.
 - 3: **repeat**
 - 4: **expectation:** Assign each point to its closest centroid.
 - 5: **maximization:** Compute the new centroid (mean) of each cluster.
 - 6: **until** The centroid positions do not change.
-

Update the centroids (initially randomly)

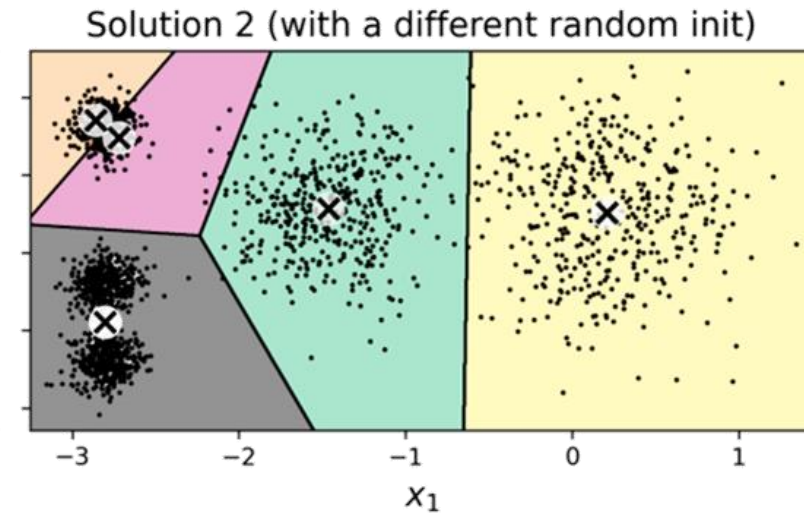
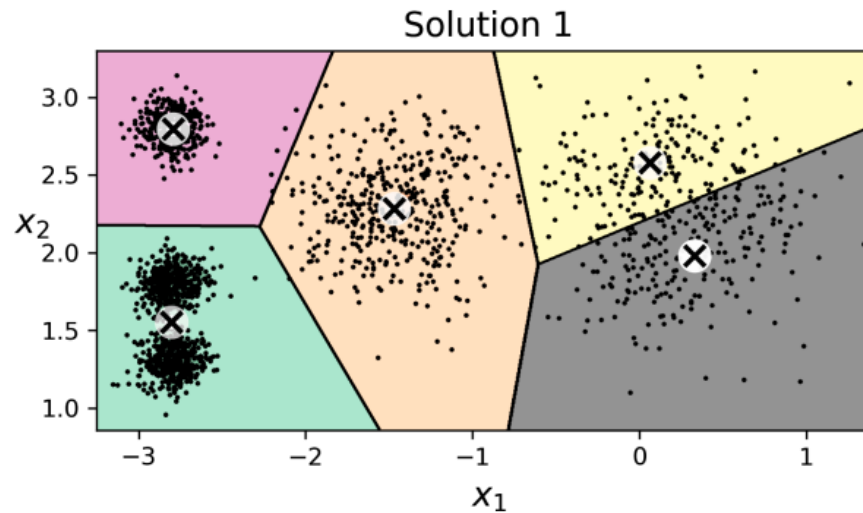
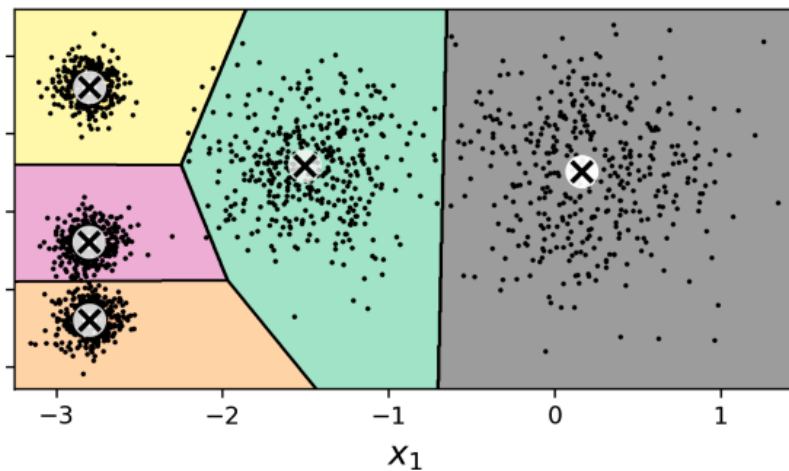


Label the instances



k-means

- اگرچه همگرایی k-means تضمین شده است، اما بسته به مراکز تصادفی اولیه، ممکن است به نتایج متفاوتی همگرا شود
- دو پاسخ غیربهمینه:



مقداردهی اولیه مراکز

- اگر به هر روشی محدوده تقریبی مراکز خوشه‌ها قابل تنظیم است، می‌توان آن را تنظیم کرد

```
good_init = np.array([[ -3, 3], [ -3, 2], [ -3, 1], [ -1, 2], [ 0, 2]])  
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1, random_state=42)  
kmeans.fit(X)
```

- می‌توان الگوریتم را چندین بار با مقادیر تصادفی متفاوت اجرا کرد و بهترین پاسخ را انتخاب کرد

- ابرپارامتر `n_init` تعداد را مشخص می‌کند که پیش‌فرض آن ۱۰ است
- کدام پاسخ بهتر است؟

▪ یکی از معیارهای مناسب، `inertia` است که برابر است با مجموع مربعات فاصله نمونه‌ها تا نزدیک‌ترین مرکز

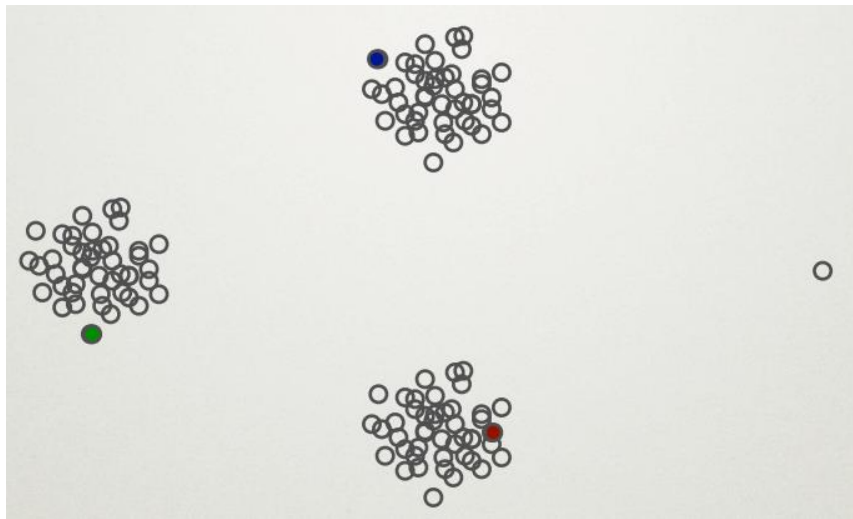
```
>>> kmeans.inertia_  
211.59853725816836
```

```
>>> kmeans.score(X)  
-211.5985372581684
```

▪ برای ۳ شکل اسلاید قبل به ترتیب برابر است با 211.6، 219.4 و 258.6

k-means++

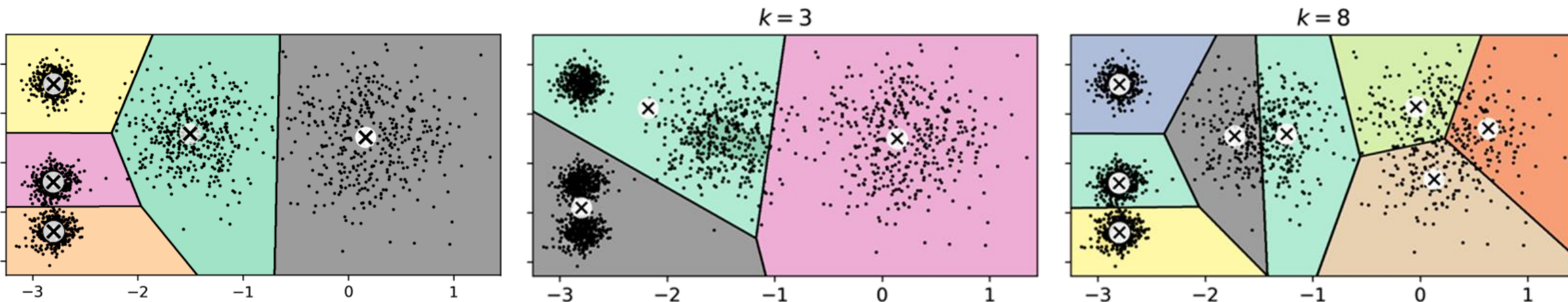
- در مقاله k-means++ پیشنهاد شده است تا مراکز اولیه به گونه‌ای انتخاب شود که فاصله آنها زیاد باشد
 - انتخاب یک نمونه $\mathbf{c}^{(1)}$ به صورت تصادفی با احتمال یکسان
 - انتخاب نمونه بعدی با احتمال $D(\mathbf{x}^{(i)})^2 / \sum_{j=1}^m D(\mathbf{x}^{(j)})^2$ که $D(\mathbf{x}^{(i)})$ فاصله $\mathbf{x}^{(i)}$ از نزدیک‌ترین مرکز موجود است
 - تکرار تا انتخاب تمام مراکز



- انتخاب مراکز بسیار بیشتر زمان می‌برد اما احتمال همگرا شدن به پاسخ suboptimal کمتر است و در مجموع می‌توان به مراتب سریعتر باشد
- کلاس Kmeans به صورت پیش‌فرض از همین مقداردهی اولیه استفاده می‌کند

انتخاب تعداد خوشه‌ها

- در بسیاری از مسائل واقعی تعداد درست خوشه‌ها را نمی‌دانیم و تعیین آن کار ساده‌ای نیست
- اگر تعداد خوشه‌ها را نامناسب قرار دهیم، ممکن است خوشه‌بندی بسیار بدی داشته باشیم
- آیا معیار inertia برای انتخاب تعداد خوشه‌ها مناسب است؟
- برای این سه خوشه‌بندی، 653.2 ($k = 3$)، 211.6 ($k = 5$) و 119.1 ($k = 8$) است
- هر چه تعداد خوشه‌ها بیشتر باشد، معیار inertia می‌تواند کمتر شود!



انتخاب تعداد خوشه‌ها

- با افزایش k تا ۴، معیار inertia به سرعت کاهش می‌یابد اما پس از آن به آهستگی کاهش می‌یابد
- این منحنی معمولاً شبیه به بازو (arm) و دارای آرنج (elbow) است که می‌تواند نقطه مناسبی باشد

